

Statistical Machine Learning (BE4M33SSU)

Stochastic Gradient Descent

Jan Drchal

Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Computer Science

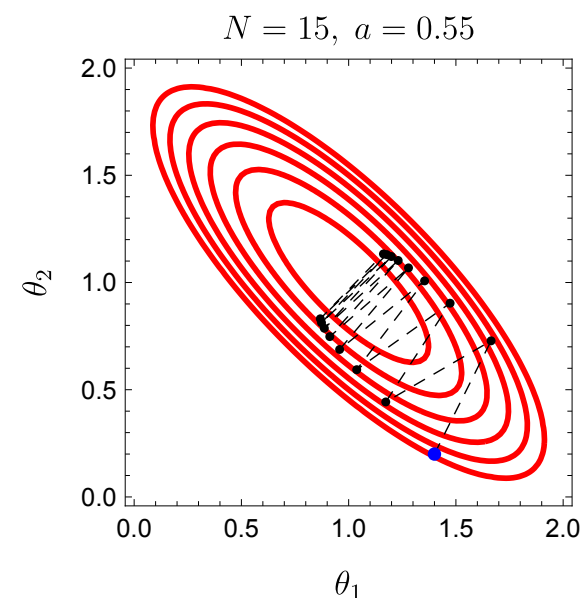
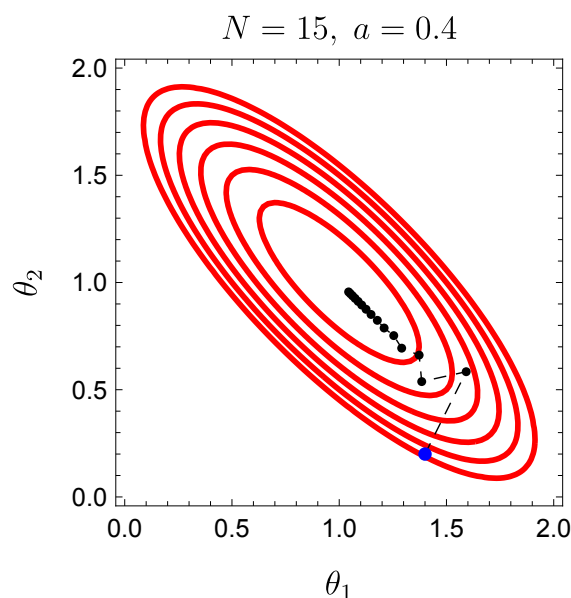
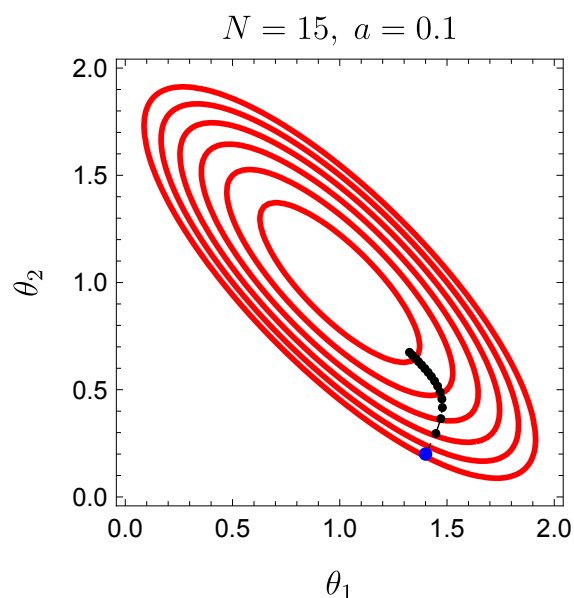
Gradient Descent (GD)

- ◆ **Task:** find parameters which minimize loss over the training dataset:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \mathcal{L}(\theta)$$

where θ is a set of all parameters defining the ANN

- ◆ Gradient descent: $\theta_{k+1} = \theta_k - \alpha_k \nabla \mathcal{L}(\theta_k)$
where $\alpha_k > 0$ is the **learning rate** or **stepsize** at iteration k



Stochastic Gradient Descent (SGD): Motivation

- ◆ The loss has typically an additive structure, hence:

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \nabla \ell(y_i, h_{\boldsymbol{\theta}}(\mathbf{x}_i))$$

- ◆ Evaluation of $\nabla \mathcal{L}(\boldsymbol{\theta})$ takes $\mathcal{O}(m)$ time
- ◆ What if we have duplicate samples in $\mathcal{T}^m$?
- ◆ Online learning, *infinite data* (augmentation)?
- ◆ Specialized loss functions (not based on likelihood)?
- ◆ Use a single sample or a *mini-batch* instead of the *full-batch* approach
 $\Rightarrow$ Stochastic Gradient Descent (SGD)
- ◆ Recommended reading: *Bottou, Curtis and Nocedal: Optimization Methods for Large-Scale Machine Learning, 2018*

SGD Algorithm

◆ Stochastic Gradient Descent

- 1 Choose an initial iterate θ_1
- 2 **for** $k = 1, 2, \dots$
- 3 Draw a *batch* $\mathcal{B}_k^M \subset \mathcal{T}^m$
- 4 Compute a stochastic gradient estimate vector $g(\theta_k)$ for $\mathcal{B}_k^M$
- 5 Choose a stepsize $\alpha_k > 0$
- 6 Set the new iterate as $\theta_{k+1} \leftarrow \theta_k - \alpha_k g(\theta_k)$

◆ The stochastic gradient estimate is defined as:

$$g(\theta_k) = \frac{1}{M} \sum_{i=1}^M \nabla \ell(y_i, h_{\theta}(\mathbf{x}_i)), \quad (\mathbf{x}_i, y_i) \in \mathcal{B}_k^M$$

Drawing Batches

- ◆ Random samples with replacement
⇒ some training samples may be left unused!
- ◆ Shuffle data once and split to batches
- ◆ **Shuffle data each epoch before splitting to batches**
- ◆ Empirical evidence: *Bottou, Curiously fast convergence of some stochastic gradient descent algorithms, 2009.*

SGD Convergence Theorem: Overview

- ◆ The main theorem shows that the *expected optimality gap*

$$\mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_k) - \mathcal{L}_*] \xrightarrow{k \rightarrow \infty} 0$$

where $\mathcal{L}_*$ is the optimal (minimal) loss

- ◆ Selected assumptions:

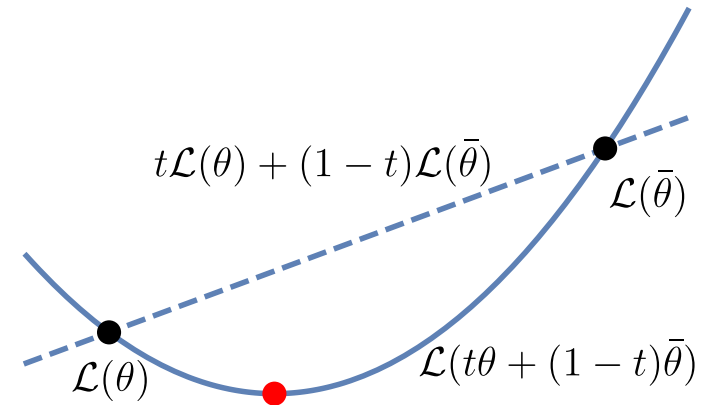
1. Strong convexity of $\mathcal{L}$
2. Lipschitz continuous gradient $\nabla \mathcal{L}$
3. Bounds on $\mathcal{L}$ and $g(\boldsymbol{\theta}_k)$:
 - directions of $g(\boldsymbol{\theta}_k)$ and $\nabla \mathcal{L}(\boldsymbol{\theta}_k)$ *similar*,
 - their norms are also *similar*

Convexity

- ◆ Convex function definition:

$$\mathcal{L}(t\boldsymbol{\theta} + (1 - t)\bar{\boldsymbol{\theta}}) \leq t\mathcal{L}(\boldsymbol{\theta}) + (1 - t)\mathcal{L}(\bar{\boldsymbol{\theta}})$$

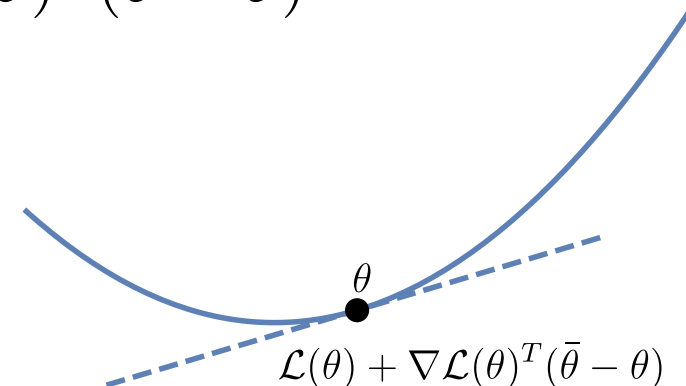
for all $(\boldsymbol{\theta}, \bar{\boldsymbol{\theta}}) \in \mathbb{R}^d \times \mathbb{R}^d$



- ◆ Equivalently (first-order condition):

$$\mathcal{L}(\bar{\boldsymbol{\theta}}) \geq \mathcal{L}(\boldsymbol{\theta}) + \nabla \mathcal{L}(\boldsymbol{\theta})^T (\bar{\boldsymbol{\theta}} - \boldsymbol{\theta})$$

the function lies above all its tangents



- ◆ See A4B33OPT
- ◆ But we need a stronger assumption...

Assumption 1: Strong Convexity

- ◆ The loss function $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ is strongly convex if there exists constant $c > 0$ such that

$$\mathcal{L}(\bar{\boldsymbol{\theta}}) \geq \mathcal{L}(\boldsymbol{\theta}) + \nabla \mathcal{L}(\boldsymbol{\theta})^T (\bar{\boldsymbol{\theta}} - \boldsymbol{\theta}) + \frac{1}{2}c \|\bar{\boldsymbol{\theta}} - \boldsymbol{\theta}\|_2^2$$

for all $(\boldsymbol{\theta}, \bar{\boldsymbol{\theta}}) \in \mathbb{R}^d \times \mathbb{R}^d$

- ◆ *Intuition:* quadratic lower bound on function growth
- ◆ Constant c **quantifies** the level of convexity $\Rightarrow$ higher c indicates "more convex" function

Assumption 2: Lipschitz Continuous Gradient

- ◆ The loss function is continuously differentiable and the gradient is *Lipschitz continuous* with *Lipschitz constant* $L > 0$:

$$\|\nabla \mathcal{L}(\boldsymbol{\theta}) - \nabla \mathcal{L}(\bar{\boldsymbol{\theta}})\|_2 \leq L \|\boldsymbol{\theta} - \bar{\boldsymbol{\theta}}\|_2, \text{ for all } (\boldsymbol{\theta}, \bar{\boldsymbol{\theta}}) \in \mathbb{R}^d \times \mathbb{R}^d$$

- ◆ *Intuition*: the gradient does not change too quickly w.r.t. $\boldsymbol{\theta}$
- ◆ Provides a **quantified** indicator for how far to move to decrease $\mathcal{L}$

Assumptions Summary

constant	description	higher value means
$c > 0$	strong convexity (lower bound)	"more convex"
$L > 0$	Lipschitz continuous gradient (upper bound)	higher gradient change allowed
$\mu > 0$	$g(\boldsymbol{\theta}_k)$ direction comparable to $\nabla \mathcal{L}(\boldsymbol{\theta}_k)$	smaller angular difference between $\mathbb{E}[g(\boldsymbol{\theta}_k)]$ and $\nabla \mathcal{L}(\boldsymbol{\theta}_k)$
$M \geq 0$	compares norms of $\mathbb{E}[g(\boldsymbol{\theta}_k)]$ and $\nabla \mathcal{L}(\boldsymbol{\theta}_k)$	higher variance in norms allowed

SGD Convergence: Strongly Convex $\mathcal{L}$, Fixed Stepsize

- ◆ **Theorem (simplified):** if the assumptions $\uparrow$ hold, the SGD is run with a fixed (and *bound*) stepsize $\alpha_k = \alpha$ for all $k \in \mathbb{N}$. Then the *expected optimality gap* satisfies the following for all k :

$$\mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_k) - \mathcal{L}_*] \leq \frac{\alpha LM}{2c\mu} + \mathcal{O}(\rho^k) \xrightarrow{k \rightarrow \infty} \frac{\alpha LM}{2c\mu},$$

where $\rho \in [0, 1)$ is a constant

- ◆ In general, for the fixed stepsize, the *optimality gap* tends to zero, but converges to $\frac{\alpha LM}{2c\mu} \geq 0$

Full-Batch Gradient Descent

- ◆ How does the theorem apply to the full-batch setting (GD)?
- ◆ The $g(\boldsymbol{\theta}_k)$ is an unbiased estimate of $\nabla \mathcal{L}(\boldsymbol{\theta}_k)$:

$$g(\boldsymbol{\theta}_k) = \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

- ◆ Zero variance implies $M = 0$, hence, $\frac{\alpha LM}{2c\mu} = 0$
- ◆ The optimality gap simplifies to:

$$\epsilon_k = \mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_k) - \mathcal{L}_*] \leq \mathcal{O}(\rho^k) \xrightarrow{k \rightarrow \infty} 0$$

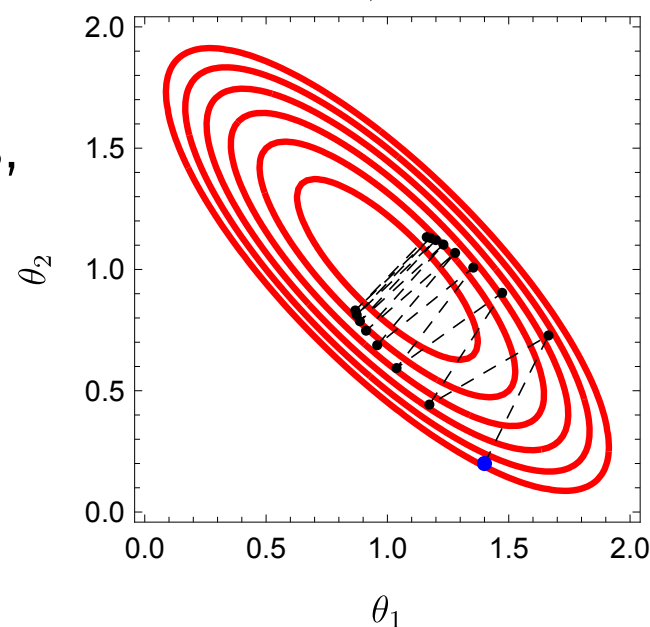
- ◆ For any given gap ϵ , the number of iterations k is proportional to $\log(1/\epsilon)$ in the worst case.

SGD Convergence: Strongly Convex $\mathcal{L}$, Diminishing Stepsize

- ◆ **Theorem (simplified):** if the assumptions $\uparrow$ hold and the SGD is run with a diminishing stepsize ($\alpha_k \approx$ inversely proportional to k). Then the *expected optimality gap* satisfies the following for all k :

$$\mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_k) - \mathcal{L}_*] \leq \mathcal{O}(1/k) \xrightarrow{k \rightarrow \infty} 0$$

- ◆ The number of iterations k is proportional to $1/\epsilon$ in the worst case
- ◆ The diminishing stepsize is needed to compensate for the noise & to prevent overshooting the minimum
- ◆ Note: in practise α_k is often reduced in steps, e.g., halved after each N epochs



SGD for Nonconvex Objectives

- ◆ Corresponding theorems can be proven for nonconvex objectives
- ◆ For assumptions similar to the theorem for the diminishing stepsizes (and excluding the strong convexity) we get:

$$\lim_{k \rightarrow \infty} \mathbb{E} \left[\|\nabla \mathcal{L}(\boldsymbol{\theta}_k)\|_2^2 \right] = 0$$

GD vs SGD

	GD	SGD
time per iteration	m	1
iterations for accuracy ϵ	$\log(1/\epsilon)$	$1/\epsilon$
time for accuracy ϵ	$m \log(1/\epsilon)$	$1/\epsilon$
error for limited time budget T_{max}	$\frac{\log(T_{max})}{T_{max}} + \frac{1}{T_{max}}$	$\frac{1}{T_{max}}$

- ◆ SGD time does not depend on dataset size (if not exhausted)
- ◆ For large-scale problems (large m) SGD is faster
- ◆ For limited time budget T_{max} SGD achieves lower error
- ◆ It is harder to tune stepsize schedule for SGD, but you can experiment on a small representative subset of the dataset
- ◆ In practise *mini-batches* are used to leverage optimization/parallelization on CPU/GPU

Momentum

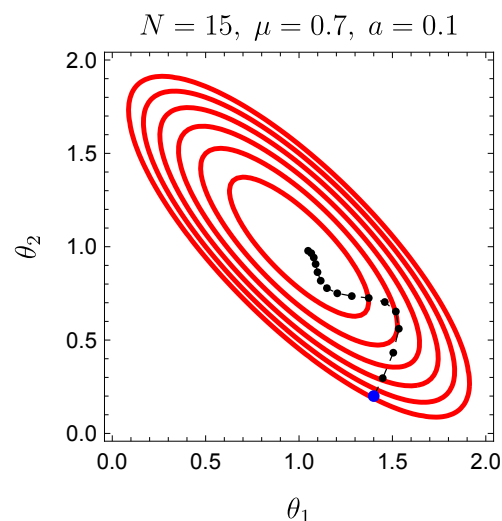
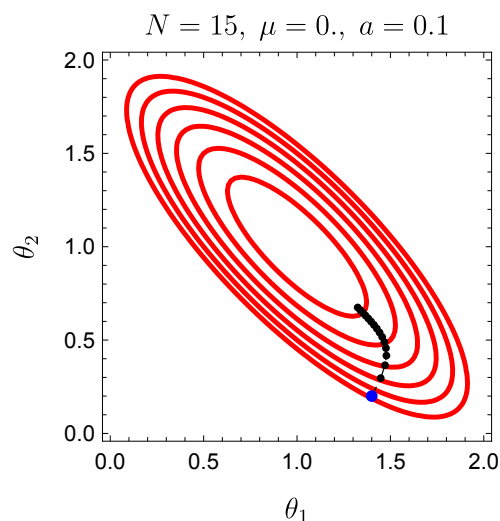
- ◆ Simulate inertia to overcome plateaus in the error landscape:

$$\mathbf{v}_{k+1} \leftarrow \mu \mathbf{v}_k - \alpha_k g(\boldsymbol{\theta}_k)$$

$$\boldsymbol{\theta}_{k+1} \leftarrow \boldsymbol{\theta}_k + \mathbf{v}_{k+1}$$

where $\mu \in [0, 1]$ is the *momentum parameter*

- ◆ Momentum damps oscillations in directions of high curvature
- ◆ It builds velocity in directions with consistent (possibly small) gradient



Adagrad

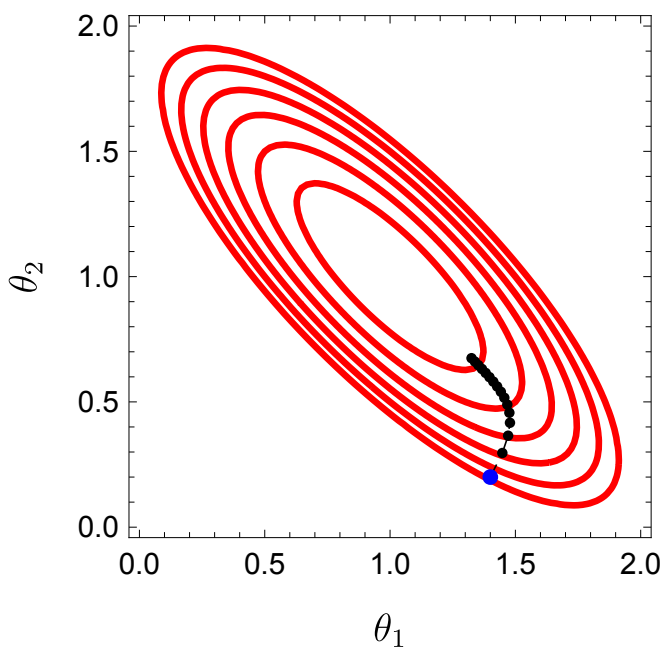
- ◆ Adaptive Gradient method (Duchi, Hazan and Singer, 2011)
- ◆ *Motivation*: a magnitude of gradient differs a lot for different parameters
- ◆ *Idea*: reduce learning rates for parameters having high values of gradient

$$G_{k+1,i} \leftarrow G_{k,i} + [g(\boldsymbol{\theta}_k)]_i^2$$

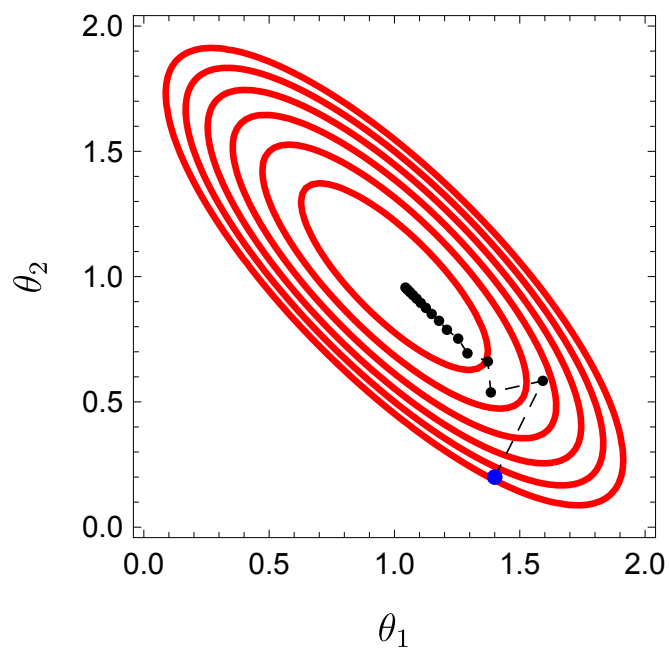
$$\theta_{k+1,i} \leftarrow \theta_{k,i} - \frac{\alpha}{\sqrt{G_{k+1,i} + \epsilon}} \cdot [g(\boldsymbol{\theta}_k)]_i$$

- ◆ $G_{k,i}$ accumulates squared partial derivative approximations w.r.t. to the parameter $\theta_{k,i}$
- ◆ ϵ is a small positive number to prevent division by zero
- ◆ *Weakness*: ever increasing G_i leads to slow convergence eventually
- ◆ Better methods: RMSProp, Adam, . . .

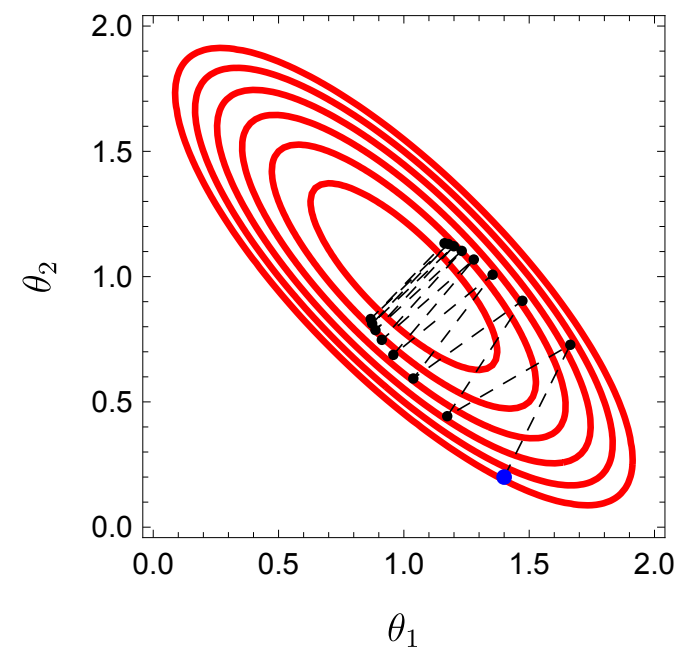
$N = 15, a = 0.1$

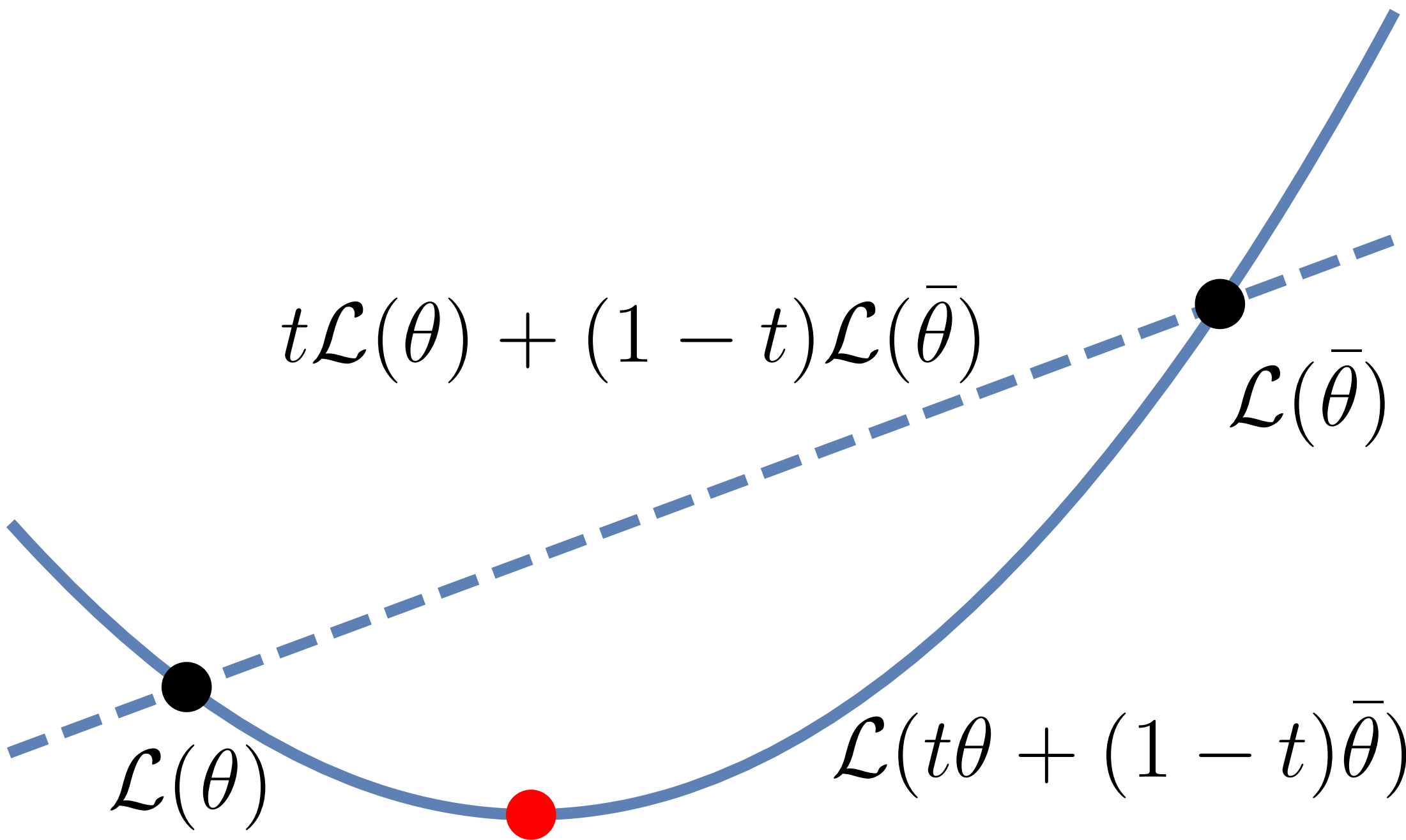


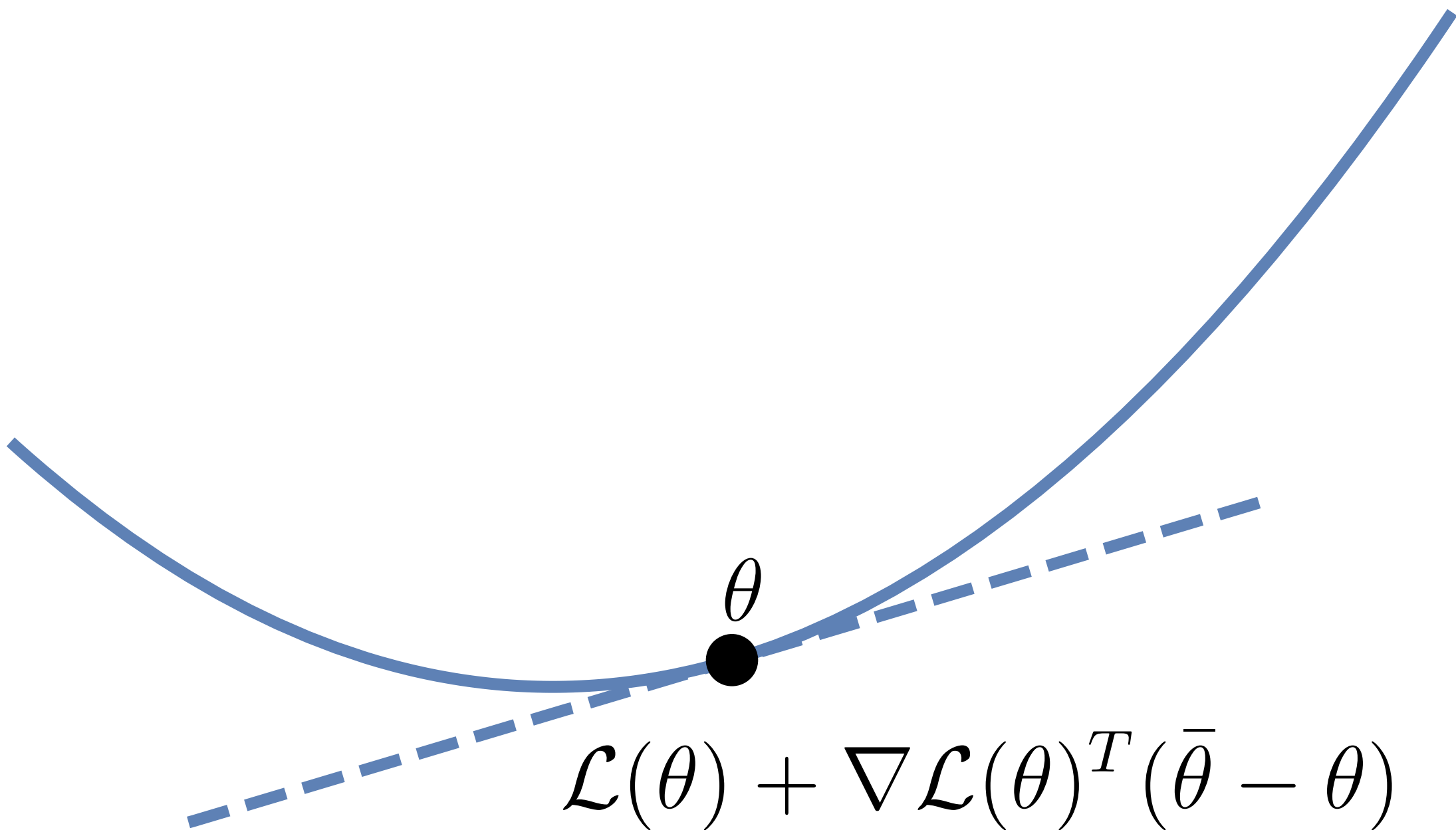
$N = 15, a = 0.4$

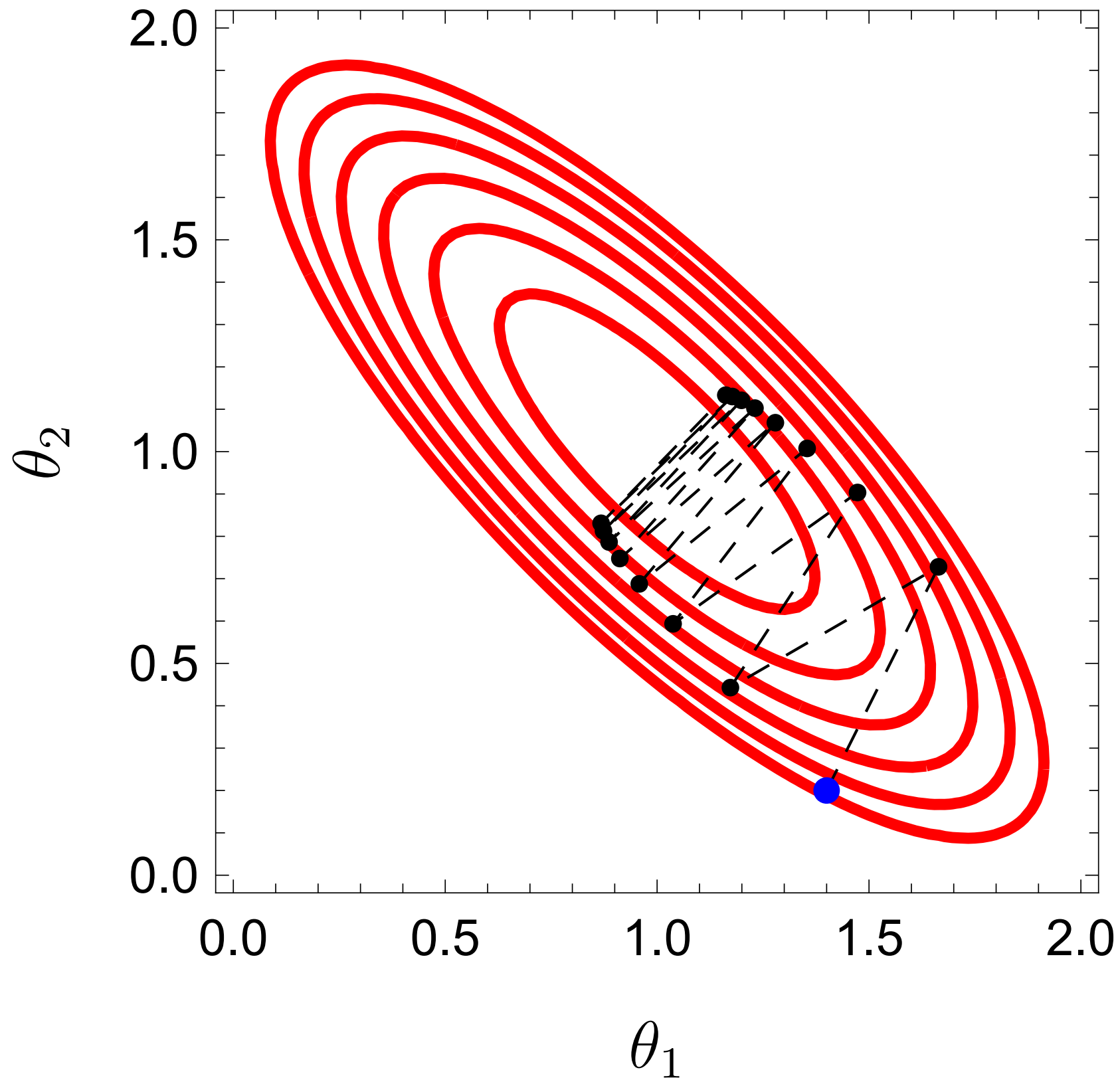


$N = 15, a = 0.55$

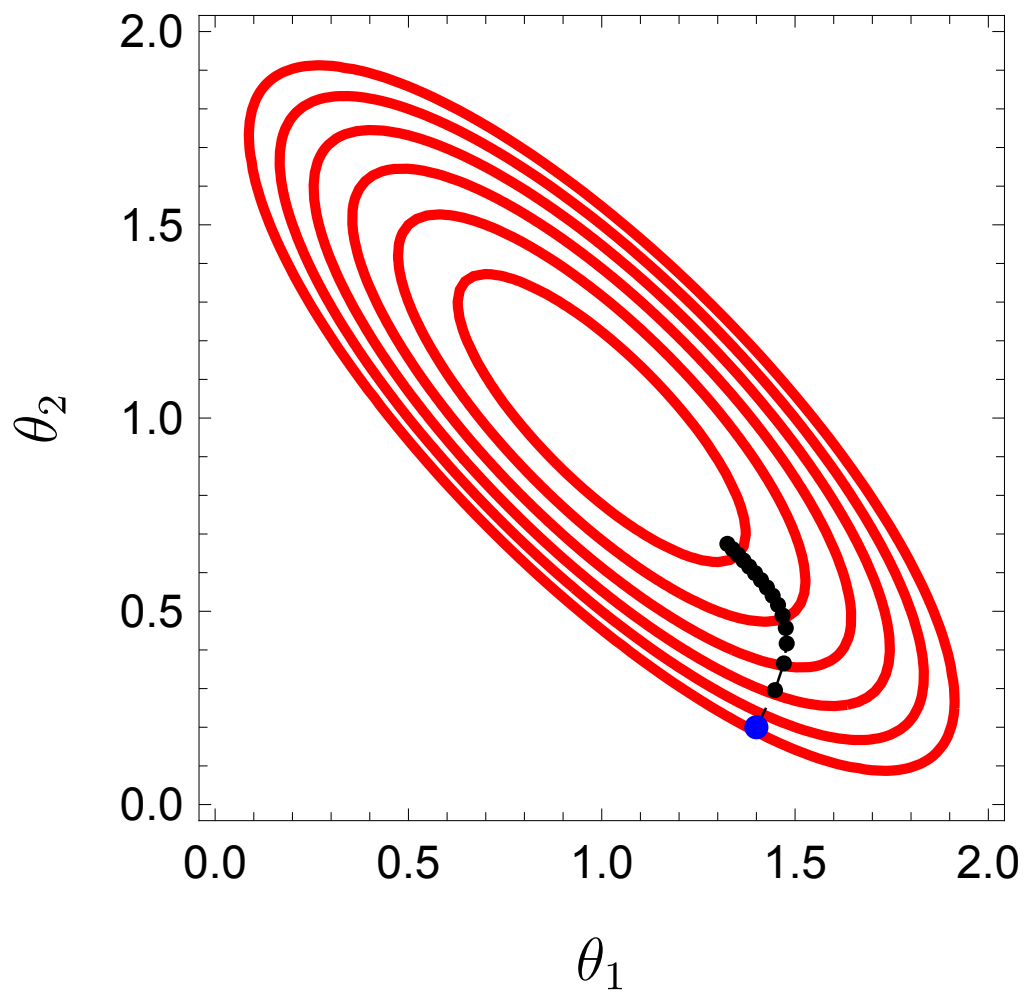








$N = 15, \mu = 0., a = 0.1$



$N = 15, \mu = 0.7, a = 0.1$

